

MANUAL DE PROGRAMADOR

ESPEJO MAGICO

Tabla de Contenido

Introducción	3
Objetivos.....	3
1.REQUERIMIENTOS DEL SISTEMA Y ENTORNO	4
2. ESTRUCTURA DEL PROYECTO Y REPOSITORIO.....	4
1.1.1 Directorio _pycache	5
1.1.2 Directorio .env.....	6
1.1.3 Directorio vs.code	6
1.1.4 Directorio artifacts	7
1.1.5 Directorio data_raw	7
1.1.6 Directorio web	8
1.2 entrenamiento tensorflow/keras.....	8
1.3 resultado y metricas.....	9
3.SERVIDOR WEB E INTERFACES DE USUARIO	9
1.3.1 Interfaces web de la camera y el entrenamiento	9
.....	10
4. ANALISIS	10
5. Conclusiones	10

INTRODUCCIÓN

El presente documento combina la documentación técnica del sistema y el informe detallado del desarrollo e implementación de un modelo de Inteligencia Artificial para el Reconocimiento de Emociones Faciales. El sistema es capaz de clasificar siete emociones principales: Angry, Disgust, Fear, Happy, Sad, Surprise y Neutral a partir de conjuntos de datos de imágenes (FER, CK+ y RAF).

Está diseñado para que el personal técnico especializado pueda comprender la estructura del código, realizar el mantenimiento, reentrenar modelos, solventar problemas e implementar despliegues tanto a nivel de servidor local/web como mediante integraciones adicionales

OBJETIVOS

- Instruir en el uso adecuado del sistema: Proveer una guía clara para la instalación, configuración del entorno, preparación de datos e inferencia en tiempo real.
- Documentar la arquitectura técnica: Detallar la lógica de cada uno de los módulos de software que componen la solución (dataset.py, model.py, train.py, evaluate.py, web_server.py, entre otros)
- Presentar resultados experimentales: Registrar las métricas del ciclo de entrenamiento y evaluar el rendimiento del modelo en producción.

1. REQUERIMIENTOS DEL SISTEMA Y ENTORNO

El sistema puede ser configurado en entornos Windows o Linux con soporte para aceleración por hardware (GPU CUDA recomendado)

Requisitos de Software y Dependencias

Lenguaje: Python 3.8+ (Recomendado Python 3.10)

- Frameworks de IA: PyTorch (con CUDA 12.6 para soporte de GPU como NVIDIA RTX 3050) o TensorFlow/Keras 2.x.
- Servidor Web Local: Flask / FastAPI.
- Librerías auxiliares: OpenCV, torchvision, scikit-learn, matplotlib, numpy, pytest.

Configuración e Instalación del Entorno

Creación del entorno virtual:

- `python -m venv venv .\venv\Scripts\Activate.ps1`

Instalación de dependencias:

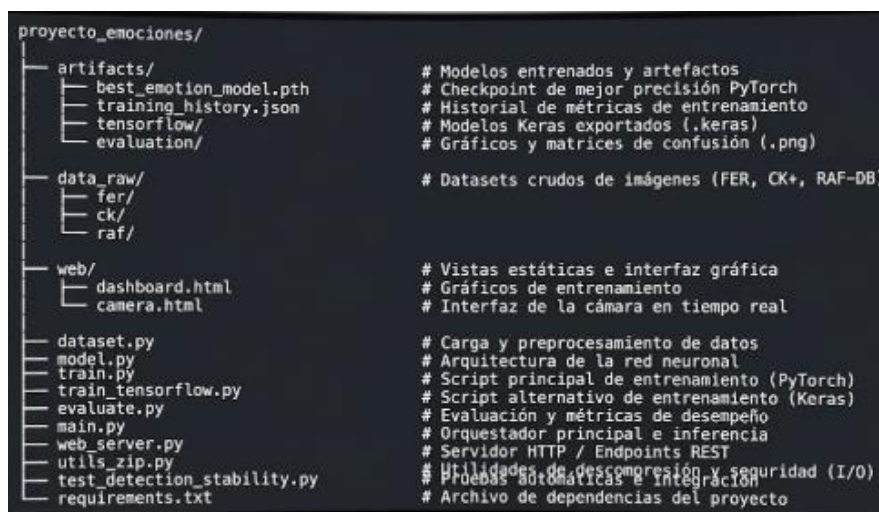
- `pip install -r requirements.txt`

Verificación de GPU (opcional):

- `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"`

2. ESTRUCTURA DEL PROYECTO Y REPOSITORIO

A continuación, se describe el árbol de directorios y la organización general del proyecto



dataset.py

- Funcionalidad: carga imágenes, aplica transformaciones y crea DataLoaders.
- Clases clave: `EmotionDataset` (o equivalente), funciones de split.

model.py

- Funcionalidad: define la red neuronal como `nn.Module`.
- Parámetros configurables: número de clases, tamaño de entrada.

train.py

- Funcionalidad: bucle de entrenamiento completo, guardado de checkpoints y `training\_history.json`.
- Parámetros: `--epochs`, `--batch\_size`, `--learning\_rate`, `--data\_path`, `--checkpoint\_path`.

train_tensorflow.py

- Funcionalidad: versión alternativa en TensorFlow (pipeline y callbacks equivalentes).

evaluate.py

- Funcionalidad: carga checkpoint y evalúa en conjunto de test, genera reportes en `artifacts/evaluation/`.

main.py

- Funcionalidad: ejemplo de inferencia sobre imagen o carpeta de imágenes.

web_server.py

- Funcionalidad: expone endpoints REST para inferencia (subida de imagen → predicción JSON / visualización).

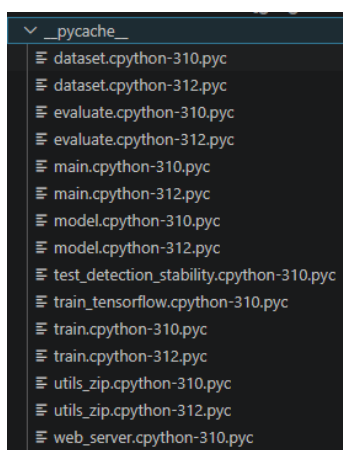
utils_zip.py

- Funcionalidad: utilidades para empaquetar/desempaquetar datasets y helpers de I/O.

test_detection_stability.py

- Funcionalidad: pruebas unitarias y de estabilidad; ejecutar con `pytest -q`.

1.1.1 Directorio _pycache



Carga y preparación de datos (dataset.py): Maneja el procesamiento y la estructuración del conjunto de datos.

Arquitectura de inteligencia artificial (model.py): Define la estructura del modelo de aprendizaje profundo o IA.

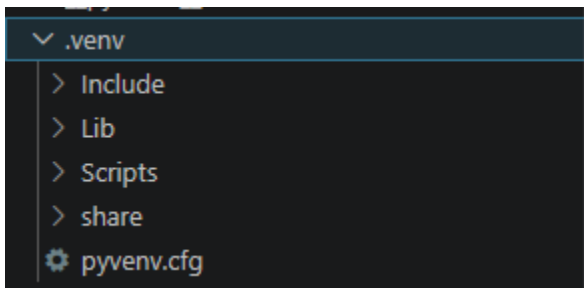
Entrenamiento (train.py y train_tensorflow.py): Scripts dedicados a entrenar el modelo, incluyendo una implementación específica en TensorFlow.

Evaluación y pruebas (evaluate.py y test_detection_stability.py): Mide el rendimiento del modelo y evalúa la estabilidad de las detecciones realizadas.

Despliegue y ejecución (main.py y web_server.py): Contiene el punto de entrada principal del proyecto y un servidor web para realizar inferencias o servir la API/interfaz del modelo.

Utilidades (utils_zip.py): Funciones auxiliares para la compresión, extracción o manejo de archivos de datos en formato ZIP.

1.1.2 Directorio .env



Lib/: Contiene las copias locales de las librerías instaladas mediante pip (por ejemplo, TensorFlow, OpenCV, etc.).

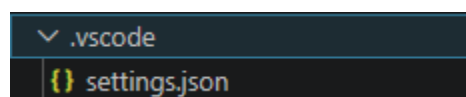
Scripts/: Guarda el ejecutable del intérprete de Python asociado al entorno, así como los comandos para activarlo/desactivarlo (activate) y herramientas como pip.

Include/: Almacena cabeceras C/C++ necesarias si alguna librería requiere compilación de código nativo.

share/: Guarda archivos de datos compartidos o documentación requerida por algunos paquetes instalados.

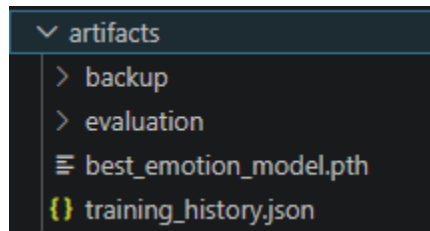
pyvenv.cfg: Archivo de configuración que define la versión de Python que utiliza el entorno y la ruta original desde donde se creó.

1.1.3 Directorio vs.code



settings.json: Define ajustes personalizados para el editor aplicables únicamente a este proyecto. Por ejemplo, suele configurar la ruta del intérprete de Python del entorno virtual (.env), reglas del formateador de código, linters, o configuraciones del editor específicas.

1.1.4 Directorio artifacts



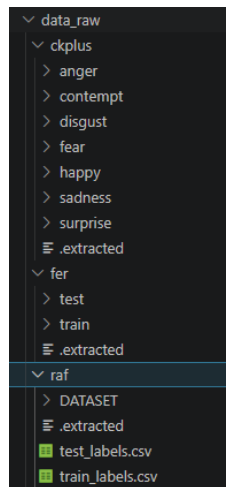
best_emotion_model.pth: El archivo con los pesos guardados del modelo entrenado en PyTorch que obtuvo el mejor rendimiento (útil para hacer inferencias o desplegar el servidor web).

training_history.json: Registro con el historial del entrenamiento (métricas como *loss*, *accuracy*, precisión por época, etc.).

evaluation/: Subcarpeta dedicada a almacenar reportes, gráficas o métricas resultantes de evaluar el modelo (evaluate.py).

backup/: Subcarpeta destinada a guardar copias de seguridad de versiones previas del modelo o de otros artefactos del proceso.

1.1.5 Directorio data_raw



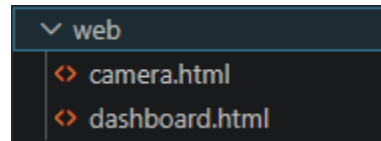
ckplus/ (Cohn-Kanade+): Organizado directamente por categorías de emociones (*anger*, *contempt*, *disgust*, *fear*, *happy*, *sadness*, *surprise*).

fer/ (FER-2013): Estructurado en divisiones tradicionales de entrenamiento y prueba (train y test).

raf/ (RAF-DB): Contiene las imágenes del conjunto (DATASET) junto con sus respectivos archivos de etiquetas en formato CSV (train_labels.csv y test_labels.csv).

Archivos. extracted: Archivos marca o *flags* generados automáticamente por el código (utils_zip.py) para confirmar que los archivos comprimidos originales de cada dataset ya fueron descomprimidos correctamente.

1.1.6 Directorio web



camera.html: Vista para capturar o transmitir video en tiempo real desde la cámara web y enviar los fotogramas al modelo para la detección de emociones en vivo.

dashboard.html: Panel de control interactivo para visualizar estadísticas, gráficas de métricas o resultados acumulados de las evaluaciones y detecciones.

1.2 ENTRENAMIENTO TENSORFLOW/KERAS

`train\_tensorflow.py` contiene el flujo equivalente en TensorFlow:

1. Descubre las imagenes etiquetadas de FER, CK+ y RAF.
2. Mezcla y separa 80% para entrenamiento y 10% para validacion.
3. Redimensiona a 224x224, normaliza a `[0, 1]` y aplica volteo horizontal al entrenamiento.
4. Usa MobileNetV2 preentrenada en ImageNet como extractor congelado.
5. Entrena una capa softmax de 7 clases con Adam y `sparse\_categorical\_crossentropy`.
6. Detiene temprano y restaura los mejores pesos.
7. Guarda `artifacts/tensorflow/emotion\_model.keras` y `artifacts/tensorflow/training\_history.json`.

Ejecutar con Python :

```
python train_tensorflow.py
```

Para el flujo PyTorch:

```
python main.py
```


1.3 RESULTADO Y METRICAS

Resultados (extraído de `artifacts/training\_history.json`)

- Framework: PyTorch
- Épocas completadas: 12
- Mejor accuracy de validación registrada: ****0.26865**** (epoch 8)
- Checkpoint guardado: `artifacts/best\_emotion\_model.pth`

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	
1	1.7426	0.1850	1.8114	0.1795	
2	1.6581	0.2420	1.7510	0.2169	
3	1.6309	0.2573	1.7516	0.2238	
4	1.6080	0.2683	1.7437	0.2163	
5	1.5877	0.2805	1.7189	0.2433	
6	1.6036	0.2849	1.7194	0.2362	
7	1.5823	0.2901	1.6960	0.2610	
8	1.5847	0.2917	1.6865	0.2686	
9	1.5767	0.2957	1.7020	0.2418	
10	1.5878	0.2889	1.6864	0.2667	
11	1.5752	0.2929	1.6937	0.2633	
12	1.5766	0.2987	1.6895	0.2683	

3.SERVIDOR WEB E INTERFACES DE USUARIO

El proyecto incluye un servidor de despliegue en tiempo real desarrollado en Python

Ejecución del Servidor Web

powershell python web_server.py

Vistas Disponibles Dashboard <http://127.0.0.1:5500/web/camera.html>

Muestra los gráficos comparativos de entrenamiento (`Loss` y `Accuracy`) y el estado actual de las métricas obtenidas desde `training\_history.json`.

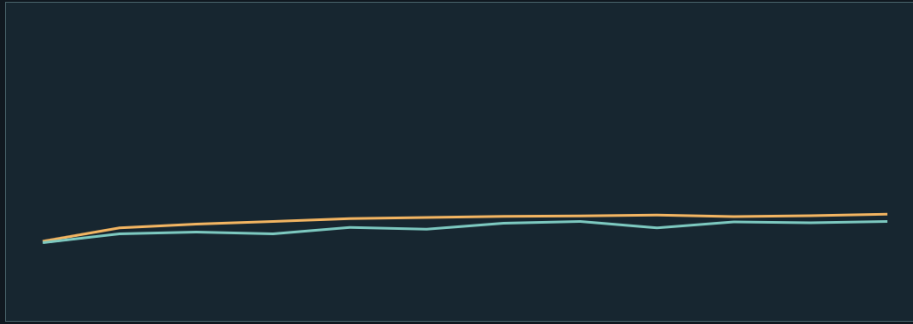
Cámara Web en Vivo <http://127.0.0.1:5500/web/dashboard.html>

1.3.1 Interfaces web de la camera y el entrenamiento



El pulso del modelo

Historial de entrenamiento, validación y mejor checkpoint.



PyTorch: 12 épocas | mejor validación: 26.86%

4. ANALISIS

El rendimiento actual alcanza un ~26.86% de precisión en validación. Este comportamiento en las métricas demuestra subajuste (*underfitting*) o estancamiento, lo cual suele originarse por:

1. Mantener la red base completamente congelada durante todas las épocas.
2. El desequilibrio en la cantidad de muestras entre emociones (ej. baja representatividad de *Disgust* en comparación con *Happy*).
3. La resolución o nivel de ruido propio de los datasets crudos

5. Conclusiones

Usar transfer learning (ResNet/efficientnet preentrenadas) y un pipeline de fine-tuning.

- Equilibrar clases y probar técnicas de oversampling/augmentation.
- Registrar experimentos con MLflow/TensorBoard y hacer tuning de hiperparámetros

Se logró diseñar e implementar de forma exitosa un sistema integral de reconocimiento de emociones faciales en tiempo real, estructurado mediante una arquitectura modular que abarca desde la preparación del dataset y el entrenamiento de modelos de visión por computadora (ResNet-18 / MobileNetV2), hasta su despliegue operativo en una interfaz web funcional (web_server.py); identificando en la fase de evaluación un rendimiento máximo de validación del 26.86%, lo que establece una base sólida para optimizaciones futuras como la aplicación de fine-tuning y aumentación avanzada de datos.

